LAPORAN PRAKTIKUM

Modul 9: HMVC, Otorisasi, dan AJAX pada React Starter Kit



Disusun Oleh:

Nama : Davin Aileenskie Sofiandy

NIM : 707012500135

Kelas : D4SIKC-49-04

Mata Kuliah : Pengembangan Aplikasi Berbasis Web

TELKOM UNIVERSITY

FAKULTAS ILMU TERAPAN

JURUSAN S1 TERAPAN SISTEM INFORMASI KOTA CERDAS

TAHUN AJARAN 2026

Perangkat Lunak	Versi / Keterangan
XAMPP (Apache & MySQL)	Terpasang pada D:\xampp
MySQL / MariaDB	Server database melalui XAMPP, database hmvc_blade
phpMyAdmin	Untuk memeriksa tabel users dan daftars
PHP	8.2.12 (ZTS Visual C++ 2019 x64)
Composer	2.10.3
Laravel Framework	12.70.2
nwidart/laravel-modules	12.0
Bootstrap 5.3, Bootstrap Icons 1.11, jQuery 3.6	Dimuat dari CDN oleh layout view
Visual Studio Code	Editor kode dan terminal project
Git	2.53.0.windows.1
Google Chrome	Untuk menjalankan aplikasi dan memeriksa request AJAX

1. Tujuan Praktikum

Praktikum Modul 9 ini saya kerjakan untuk membangun aplikasi Laravel dengan struktur HMVC, pembatasan akses berdasarkan role, dan CRUD berbasis AJAX di atas Laravel React Starter Kit. Materi modul memakai aplikasi perpustakaan, sedangkan project yang saya pakai adalah pertemuan9_hmvc_react dengan database hmvc_react. Starter kit sudah membawa Inertia, React, TypeScript, Tailwind, dan shadcn/ui, jadi hampir semua tampilan ditulis sebagai komponen React, bukan Blade. Tujuan yang ingin saya capai adalah:

1. Memasang nwidart/laravel-modules pada starter kit React dan membuat dua modul, yaitu UserManagement dan Pendaftaran.

2. Menambahkan kolom role pada tabel users, membuat enum UserRole, dan memasang middleware role pada route.

3. Memakai autentikasi bawaan starter kit (register, login, logout) yang berada di luar modul, lalu memastikan akun baru selalu beroleh role user.

4. Membuat modul UserManagement dengan CRUD akun yang hanya bisa dibuka admin, lengkap dengan pencarian dan filter role.

5. Membuat modul Pendaftaran dengan CRUD berbasis AJAX, respons JSON, dan unggah berkas lewat FormData.

6. Menampilkan pesan berhasil dan pesan error per input, baik dari flash session maupun dari respons 422 JSON.

7. Menulis pengujian otomatis dengan Pest untuk role, middleware, CRUD kedua modul, dan seeder.

2. Alat dan Bahan

Praktikum ini saya kerjakan pada laptop pribadi dengan Windows 11 Pro, prosesor Intel Core i7-13650HX, dan RAM 16 GB. Berkas project disimpan di D:\xampp\htdocs\PABW\Praktikum_9\pertemuan9_hmvc_react dan dijalankan lewat XAMPP (Apache dan MySQL) serta php artisan serve.

2.1 Perangkat Lunak

Versi perangkat lunak di bawah ini saya catat langsung dari mesin yang dipakai, bukan dari dokumentasi. PHP 8.2.12, Composer 2.10.3, Node.js v25.6.0, dan npm 11.8.0. Laravel yang terpasang adalah laravel/framework v12.71.0 dengan inertiajs/inertia-laravel v2.0.28. Paket modularnya nwidart/laravel-modules v12.0.5. Pengujian memakai pestphp/pest v3.8.7 dan pest-plugin-laravel v3.2.0. Formatter kode memakai laravel/pint v1.30.4.

Untuk tampilan, starter kit memakai React 19 dengan TypeScript, Vite, Tailwind CSS, dan shadcn/ui. Ikon diambil dari lucide-react dan helper route dari Ziggy. Tidak ada jQuery atau Bootstrap seperti pada modul versi Blade, karena semua halaman dirender oleh React melalui Inertia.

2.2 Bahan

Bahannya adalah Laravel React Starter Kit yang sudah terpasang beserta autentikasinya. Route login, register, logout, verifikasi email, dan reset password ada di routes/auth.php dengan controller di app/Http/Controllers/Auth. Layout aplikasi, sidebar, dan komponen shadcn/ui sudah tersedia, jadi yang saya tambahkan adalah kolom role, middleware, dua modul, dan halaman React untuk tiap modul. Database hmvc_react dibuat lewat phpMyAdmin dengan pengguna root tanpa password mengikuti konfigurasi XAMPP.

3. Langkah-Langkah Praktikum

Urutan pengerjaan saya mulai dari database dan konfigurasi, lalu role, lalu dua modul. Modul UserManagement saya kerjakan lebih dulu karena halaman adminnya memakai cara Inertia biasa. Setelah itu modul Pendaftaran yang seluruh operasinya lewat AJAX, dan terakhir pengujian otomatis.

3.1 Menyiapkan Database dan Memasang Paket Modul

Isi .env saya sesuaikan dengan database lokal: DB_CONNECTION=mysql, DB_HOST=127.0.0.1, DB_PORT=3306, DB_DATABASE=hmvc_react, DB_USERNAME=root, dan DB_PASSWORD kosong. Karena memakai starter kit React, perintah php artisan migrate dijalankan setelah npm install dan npm run build, supaya aset Vite tersedia. Paket modular dipasang dengan composer require nwidart/laravel-modules lalu php artisan vendor:publish --provider="Nwidart\Modules\LaravelModulesServiceProvider".

Dua modul dibuat dengan php artisan module:make UserManagement dan php artisan module:make Pendaftaran. Hasilnya berupa folder Modules/UserManagement dan Modules/Pendaftaran yang masing-masing punya app, routes, resources, dan database sendiri. Status modul dicatat di modules_statuses.json.



3.2 Menambahkan Kolom Role dan Enum UserRole

Kolom role saya tambahkan lewat migration baru, bukan dengan mengubah migration users bawaan, supaya riwayat migration tetap rapi. Kolomnya berupa string dengan nilai bawaan user, jadi akun lama otomatis menjadi user biasa. Method down menghapus kolom itu kembali.

Untitled-1

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::table('users', function (Blueprint $table) {
            $table->string('role')->default('user')->after('email');
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::table('users', function (Blueprint $table) {
            $table->dropColumn('role');
        });
    }
};
```

Nilai role tidak saya tulis sebagai string bebas di banyak tempat. Saya membuat enum UserRole di app/Enums/UserRole.php dengan dua case, Admin dan User, serta method label() untuk teks yang ditampilkan di tampilan. Backed enum seperti ini bisa di-cast langsung oleh Eloquent dan divalidasi dengan Rule::enum().

Untitled-1

```php
<?php

namespace App\Enums;

enum UserRole: string
{
    case Admin = 'admin';
    case User = 'user';

    /**
     * Get the human readable label of the role.
     */
    public function label(): string
    {
        return match ($this) {
            self::Admin => 'Admin',
            self::User => 'User',
        };
    }
}
```

3.3 Model User

Model User saya ubah di tiga tempat. Kolom role masuk ke $fillable, nilai bawaannya diatur user lewat $attributes, dan $casts mengubahnya menjadi enum UserRole. Saya juga menambahkan dua method: isAdmin() untuk mengecek role admin, dan hasRole() yang menerima enum maupun string sehingga bisa dipakai middleware dengan parameter role:admin,user.

Untitled-1

```php
<?php

namespace App\Models;

// use Illuminate\Contracts\Auth\MustVerifyEmail;
use App\Enums\UserRole;
use Database\Factories\UserFactory;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Foundation\Auth\User as Authenticatable;
use Illuminate\Notifications\Notifiable;

class User extends Authenticatable
{
    /** @use HasFactory<UserFactory> */
    use HasFactory, Notifiable;

    /**
     * The attributes that are mass assignable.
     *
     * @var list<string>
     */
    protected $fillable = [
        'name',
        'email',
        'role',
        'password',
    ];

    /**
     * The attributes that should be hidden for serialization.
     *
     * @var list<string>
     */
    protected $hidden = [
        'password',
        'remember_token',
    ];

    /**
     * The model's default attribute values.
     *
     * @var array<string, mixed>
     */
    protected $attributes = [
        'role' => 'user',
    ];

    /**
     * Get the attributes that should be cast.
     *
     * @return array<string, string>
     */
    protected function casts(): array
    {
        return [
            'email_verified_at' => 'datetime',
            'password' => 'hashed',
            'role' => UserRole::class,
        ];
    }

    /**
     * Determine whether the user has the admin role.
     */
    public function isAdmin(): bool
    {
        return $this->role === UserRole::Admin;
    }

    /**
     * Determine whether the user has one of the given roles.
     */
    public function hasRole(UserRole|string ...$roles): bool
    {
        foreach ($roles as $role) {
            if ($this->role === ($role instanceof UserRole ? $role :
UserRole::tryFrom($role))) {
                return true;
            }
        }

        return false;
```

Selama pengerjaan, starter kit ini sempat membawa kolom two_factor_secret dan two_factor_recovery_codes di model dan factory, padahal migration users tidak punya kolom tersebut. Karena fitur dua faktor tidak dipakai di modul ini, bagian itu saya hapus dari model, factory, dan trait TwoFactorAuthenticatable. Setelah dihapus, migrate:fresh --seed berjalan tanpa error. Factory User saya beri state admin() supaya seeder dan pengujian lebih mudah menulis data admin.

3.4 Middleware EnsureUserHasRole

Pembatasan akses saya taruh di satu middleware bernama EnsureUserHasRole. Middleware ini menerima daftar role lewat parameter variadic, jadi penulisannya bisa role:admin atau role:admin,user. Kalau pengguna belum login atau role-nya tidak cocok, request dihentikan dengan 403 beserta pesan singkat berbahasa Indonesia.

Untitled-1

```php
<?php

namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
use Symfony\Component\HttpFoundation\Response;

class EnsureUserHasRole
{
    /**
     * Ensure the authenticated user has one of the given roles.
     *
     * Usage: ->middleware('role:admin') or ->middleware('role:admin,user')
     *
     * @param  Closure(Request): (Response)  $next
     */
    public function handle(Request $request, Closure $next, string ...$roles): Response
    {
        $user = $request->user();
        if ($user === null || ! $user->hasRole(...$roles)) {
            abort(403, 'Anda tidak memiliki hak akses ke halaman ini.');
        }

        return $next($request);
    }
}
```

Middleware didaftarkan di bootstrap/app.php dengan alias role. Di file yang sama, HandleInertiaRequests sudah dipasang pada grup web bawaan starter kit, jadi data pengguna dan flash message bisa dibagikan ke semua halaman React.

Untitled-1

```php
<?php

use App\Http\Middleware\EnsureUserHasRole;
use App\Http\Middleware\HandleAppearance;
use App\Http\Middleware\HandleInertiaRequests;
use Illuminate\Foundation\Application;
use Illuminate\Foundation\Configuration\Exceptions;
use Illuminate\Foundation\Configuration\Middleware;
use Illuminate\Http\Middleware\AddLinkHeadersForPreloadedAssets;

return Application::configure(basePath: dirname(__DIR__))
    ->withRouting(
        web: __DIR__.'/../routes/web.php',
        commands: __DIR__.'/../routes/console.php',
        health: '/up',
    )
    ->withMiddleware(function (Middleware $middleware): void {
        $middleware->encryptCookies(except: ['appearance', 'sidebar_state']);

        $middleware->web(append: [
            HandleAppearance::class,
            HandleInertiaRequests::class,
            AddLinkHeadersForPreloadedAssets::class,
        ]);

        $middleware->alias([
            'role' => EnsureUserHasRole::class,
        ]);
    })
    ->withExceptions(function (Exceptions $exceptions): void {
        //
    })->create();
```

3.5 Controller Modul UserManagement

Modul UserManagement dipakai admin untuk mengelola akun. Method index membaca parameter search dan role, lalu menyusun query dengan when() supaya filter hanya aktif kalau diisi. Hasilnya dipaginasi sepuluh baris dan diubah lewat through() menjadi array kecil yang aman dikirim ke React, lengkap dengan role_label dan created_at dalam bentuk diffForHumans().

Untitled-1

```php
<?php

namespace Modules\UserManagement\Http\Controllers;

use App\Enums\UserRole;
use App\Http\Controllers\Controller;
use App\Models\User;
use Illuminate\Database\Eloquent\Builder;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
use Inertia\Inertia;
use Inertia\Response;
use Modules\UserManagement\Http\Requests\StoreUserRequest;
use Modules\UserManagement\Http\Requests\UpdateUserRequest;

class UserManagementController extends Controller
{
    /**
     * Display a listing of users, optionally filtered by keyword and role.
     */
    public function index(Request $request): Response
    {
        $search = $request->string('search')->trim()->toString();
        $role = UserRole::tryFrom($request->string('role')->toString());

        $users = User::query()
            ->when($search !== '', function (Builder $query) use ($search) {
                $query->where(function (Builder $query) use ($search) {
                    $query->where('name', 'like', "%{$search}%")
                        ->orWhere('email', 'like', "%{$search}%");
                });
            })
            ->when($role, fn (Builder $query) => $query->where('role', $role))
            ->orderBy('name')
            ->paginate(10)
            ->withQueryString()
            ->through(fn (User $user) => [
                'id' => $user->id,
                'name' => $user->name,
                'email' => $user->email,
                'role' => $user->role->value,
                'role_label' => $user->role->label(),
                'created_at' => $user->created_at?->diffForHumans(),
            ]);

        return Inertia::render('UserManagement::index', [
            'users' => $users,
            'filters' => [
                'search' => $search,
                'role' => $role?->value ?? '',
            ],
            'roles' => $this->roleOptions(),
        ]);
    }

    /**
     * Show the form for creating a new user.
     */
    public function create(): Response
    {
        return Inertia::render('UserManagement::create', [
            'roles' => $this->roleOptions(),
        ]);
    }

```

Method update hanya mengubah password kalau kolomnya diisi, karena nilai password kosong dibuang dari data tervalidasi. Method destroy menolak penghapusan akun sendiri supaya admin tidak mengunci dirinya keluar, dan mengembalikan pesan error lewat flash. Daftar pilihan role untuk select di React diambil dari UserRole::cases() lewat roleOptions(), jadi menambah role baru cukup dilakukan di enum.

3.6 Route dan Validasi Form User

Route modul UserManagement memakai Route::resource users dengan prefix admin dan nama admin., dibungkus middleware auth serta role:admin. Method show tidak dipakai sehingga dikecualikan. Karena semuanya berada di grup itu, URL /admin/users langsung mengembalikan 403 bagi user biasa.

Untitled-1

```php
<?php

use Illuminate\Support\Facades\Route;
use Modules\UserManagement\Http\Controllers\UserManagementController;

Route::middleware(['auth', 'role:admin'])
    ->prefix('admin')
    ->name('admin.')
    ->group(function () {
        Route::resource('users', UserManagementController::class)->except('show');
    });
```

Validasi form tambah user ada di StoreUserRequest. Name dan email wajib, email harus unik di tabel users, role wajib sesuai enum, dan password wajib dikonfirmasi serta mengikuti aturan Password::defaults(). Authorize() hanya mengizinkan pengguna dengan role admin, jadi request langsung ditolak walaupun route terlewat.

Untitled-1

```php
<?php

namespace Modules\UserManagement\Http\Requests;

use App\Enums\UserRole;
use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Validation\Rule;
use Illuminate\Validation\Rules\Password;

class StoreUserRequest extends FormRequest
{
    /**
     * Get the validation rules that apply to the request.
     *
     * @return array<string, array<int, mixed>>
     */
    public function rules(): array
    {
        return [
            'name' => ['required', 'string', 'max:255'],
            'email' => ['required', 'string', 'email', 'max:255', 'unique:users,email'],
            'role' => ['required', Rule::enum(UserRole::class)],
            'password' => ['required', 'confirmed', Password::defaults()],
        ];
    }

    /**
     * Determine if the user is authorized to make this request.
     */
    public function authorize(): bool
    {
        return $this->user()?->isAdmin() ?? false;
    }
}
```

UpdateUserRequest hampir sama, tetapi email diabaikan untuk akun yang sedang diedit dan password boleh kosong. Ada tambahan Rule::when() yang memaksa role tetap admin kalau yang diedit adalah akun sendiri, dengan pesan Anda tidak dapat menurunkan role akun Anda sendiri.

Untitled-1

```php
<?php

namespace Modules\UserManagement\Http\Requests;

use App\Enums\UserRole;
use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Validation\Rule;
use Illuminate\Validation\Rules\Password;

class UpdateUserRequest extends FormRequest
{
    /**
     * Get the validation rules that apply to the request.
     *
     * An admin cannot demote their own account, so they never lock themselves out.
     *
     * @return array<string, array<int, mixed>>
     */
    public function rules(): array
    {
        $user = $this->route('user');
        $isEditingOwnAccount = $user?->is($this->user()) ?? false;

        return [
            'name' => ['required', 'string', 'max:255'],
            'email' => ['required', 'string', 'email', 'max:255', Rule::unique('users', 'email')->ignore($user?->id)],
            'role' => [
                'required',
                Rule::enum(UserRole::class),
                Rule::when($isEditingOwnAccount, Rule::in([UserRole::Admin->value])),
            ],
            'password' => ['nullable', 'confirmed', Password::defaults()],
        ];
    }

    /**
     * Get custom messages for validator errors.
     *
     * @return array<string, string>
     */
    public function messages(): array
    {
        return [
            'role.in' => 'Anda tidak dapat menurunkan role akun Anda sendiri.',
        ];
    }

    /**
     * Determine if the user is authorized to make this request.
     */
    public function authorize(): bool
    {
        return $this->user()?->isAdmin() ?? false;
    }
}
```

3.7 Tampilan React Modul UserManagement

Form tambah dan edit sama-sama memakai komponen user-form.tsx. Komponen ini menerima data, errors, processing, roles, dan setData dari useForm() Inertia, sehingga tidak ada logika form yang terduplikasi di dua halaman. Pesan error per input ditampilkan lewat komponen InputError yang bawaan starter kit.

Untitled-1

```tsx
import { Link } from '@inertiajs/react';
import type { FormEvent } from 'react';
import InputError from '@/components/input-error';
import { Button } from '@/components/ui/button';
import { Input } from '@/components/ui/input';
import { Label } from '@/components/ui/label';

export type UserFormData = {
    name: string;
    email: string;
    role: string;
    password: string;
    password_confirmation: string;
};

type UserFormProps = {
    data: UserFormData;
    errors: Partial<Record<keyof UserFormData, string>>;
    processing: boolean;
    roles: { value: string; label: string }[];
    isEdit?: boolean;
    submitLabel: string;
    setData: (key: keyof UserFormData, value: string) => void;
    onSubmit: (event: FormEvent) => void;
};

export function UserForm({
    data,
    errors,
    processing,
    roles,
    isEdit = false,
    submitLabel,
    setData,
    onSubmit,
}: UserFormProps) {
    return (
        <form onSubmit={onSubmit} noValidate className="max-w-xl space-y-5 rounded-xl
border p-6">
            <div className="grid gap-2">
                <Label htmlFor="name">
                    Nama <span className="text-red-500">*</span>
                </Label>
                <Input
                    id="name"
                    value={data.name}
                    onChange={(event) => setData('name', event.target.value)}
                    autoFocus
                />
                <InputError message={errors.name} />
            </div>

            <div className="grid gap-2">
                <Label htmlFor="email">
                    Email <span className="text-red-500">*</span>
                </Label>
                <Input
                    id="email"
                    type="email"
                    value={data.email}
                    onChange={(event) => setData('email', event.target.value)}
                />
                <InputError message={errors.email} />
            </div>

```

Halaman index menampilkan tabel akun, kotak pencarian, dan select filter role. Pencarian dan filter dikirim ulang lewat router.get() dengan preserveState dan replace, jadi halaman tidak dimuat ulang penuh. Tombol hapus membuka ConfirmDialog sebelum router.delete() dijalankan, dan akun yang sedang login tidak menampilkan tombol hapus.

Untitled-1

```tsx
import { Head, Link, router } from '@inertiajs/react';
import { Pencil, Plus, Search, Trash2 } from 'lucide-react';
import { useState } from 'react';
import type { FormEvent } from 'react';
import { FlashMessage } from '@/components/alert-message';
import { ConfirmDialog } from '@/components/confirm-dialog';
import { Badge } from '@/components/ui/badge';
import { Button } from '@/components/ui/button';
import { Input } from '@/components/ui/input';
import { useSharedProps } from '@/hooks/use-shared-props';
import AppLayout from '@/layouts/app-layout';
import type { BreadcrumbItem } from '@/types';

type UserRow = {
    id: number;
    name: string;
    email: string;
    role: 'admin' | 'user';
    role_label: string;
    created_at: string | null;
};

type PaginationLink = { url: string | null; label: string; active: boolean };

type IndexProps = {
    users: {
        data: UserRow[];
        links: PaginationLink[];
        from: number | null;
        to: number | null;
        total: number;
    };
    filters: { search: string; role: string };
    roles: { value: string; label: string }[];
};

const breadcrumbs: BreadcrumbItem[] = [{ title: 'Kelola User', href: '/admin/users' }];

export default function Index({ users, filters, roles }: IndexProps) {
    const { user: currentUser } = useSharedProps();
    const [search, setSearch] = useState(filters.search);
    const [deleting, setDeleting] = useState<UserRow | null>(null);
    const [processing, setProcessing] = useState(false);

    // Kirim ulang request GET /admin/users dengan kata kunci dan filter role
    const applyFilters = (params: { search?: string; role?: string }) => {
        router.get(
            '/admin/users',
            { ...filters, search, ...params },
            {
                preserveState: true,
                replace: true,
            },
        );
    };

    const handleSearch = (event: FormEvent) => {
        event.preventDefault();
        applyFilters({ search });
    };

    const handleDelete = () => {
        if (!deleting) {
            return;
        }
```

Halaman create memakai useForm() dengan nilai awal role user. Saat form dikirim, post() mengarah ke /admin/users, lalu kalau berhasil Inertia mengikuti redirect dari controller menuju halaman daftar beserta flash success.

Untitled-1

```tsx
import { Head, useForm } from '@inertiajs/react';
import type { FormEvent } from 'react';
import AppLayout from '@/layouts/app-layout';
import type { BreadcrumbItem } from '@/types';
import { UserForm } from '../components/user-form';
import type { UserFormData } from '../components/user-form';

type CreateProps = {
    roles: { value: string; label: string }[];
};

const breadcrumbs: BreadcrumbItem[] = [
    { title: 'Kelola User', href: '/admin/users' },
    { title: 'Tambah User', href: '/admin/users/create' },
];

export default function Create({ roles }: CreateProps) {
    const { data, setData, post, processing, errors } = useForm<UserFormData>({
        name: '',
        email: '',
        role: 'user',
        password: '',
        password_confirmation: '',
    });

    const submit = (event: FormEvent) => {
        event.preventDefault();
        post('/admin/users');
    };

    return (
        <AppLayout breadcrumbs={breadcrumbs}>
            <Head title="Tambah User" />

            <div className="flex flex-1 flex-col gap-4 p-4">
                <div>
                    <h1 className="text-xl font-semibold">Tambah User</h1>
                    <p className="text-sm text-muted-foreground">Isi data akun baru
beserta role-nya.</p>
                </div>

                <UserForm
                    data={data}
                    errors={errors}
                    processing={processing}
                    roles={roles}
                    submitLabel="Simpan"
                    setData={setData}
                    onSubmit={submit}
                />
            </div>
        </AppLayout>
    );
}
```

3.8 Controller dan Route Modul Pendaftaran

Modul Pendaftaran dibungkus middleware auth dan role:admin,user, jadi admin maupun user biasa bisa mengaksesnya. Rutenya dipisah antara menampilkan halaman dan mengambil data, agar AJAX bisa meminta data tanpa memuat komponen React lagi. Update memakai PUT dengan parameter {daftar} yang di-resolve otomatis menjadi model Daftar.

Untitled-1

```php
<?php

use Illuminate\Support\Facades\Route;
use Modules\Pendaftaran\Http\Controllers\PendaftaranController;

Route::middleware(['auth', 'role:admin,user'])->group(function () {
    Route::get('pendaftaran', [PendaftaranController::class, 'index'])->name('pendaftaran.index');

    Route::get('pendaftaran/data', [PendaftaranController::class, 'getData'])->name('pendaftaran.data');

    Route::post('pendaftaran', [PendaftaranController::class, 'store'])->name('pendaftaran.store');

    Route::get('pendaftaran/{daftar}/edit', [PendaftaranController::class, 'edit'])->name('pendaftaran.edit');

    Route::put('pendaftaran/{daftar}', [PendaftaranController::class, 'update'])->name('pendaftaran.update');

    Route::delete('pendaftaran/{daftar}', [PendaftaranController::class, 'destroy'])->name('pendaftaran.destroy');
});
```

PendaftaranController mengembalikan halaman React pada index, sedangkan getData() mengirim seluruh data sebagai JSON. Method store menyimpan berkas dengan store('berkas_pendaftaran', 'public') dan mencatat nama asli berkas untuk ditampilkan di tabel. Method update hanya mengganti berkas kalau ada yang diunggah, dan berkas lama di disk public dihapus sebelum berkas pengganti disimpan.

Untitled-1

```php
<?php

namespace Modules\Pendaftaran\Http\Controllers;

use App\Http\Controllers\Controller;
use Illuminate\Http\JsonResponse;
use Illuminate\Support\Facades\Storage;
use Inertia\Inertia;
use Inertia\Response;
use Modules\Pendaftaran\Http\Requests\DaftarRequest;
use Modules\Pendaftaran\Models\Daftar;

class PendaftaranController extends Controller
{
    /**
     * Display the React page. Data is loaded via AJAX.
     */
    public function index(): Response
    {
        return Inertia::render('Pendaftaran::index');
    }

    /**
     * Return all registrations as JSON.
     */
    public function getData(): JsonResponse
    {
        return response()->json(Daftar::latest()->get());
    }

    /**
     * Store a newly created registration together with its uploaded file.
     */
    public function store(DaftarRequest $request): JsonResponse
    {
        $data = $request->safe()->except('berkas');

        $file = $request->file('berkas');
        $data['berkas'] = $file->store('berkas_pendaftaran', 'public');
        $data['nama_berkas'] = $file->getClientOriginalName();

        Daftar::create($data);

        return response()->json(['success' => 'Data berhasil disimpan.']);
    }

    /**
     * Return a single registration for the edit form.
     */
    public function edit(Daftar $daftar): JsonResponse
    {
        return response()->json($daftar);
    }

    /**
     * Update the specified registration. The old file is replaced only when a new one is uploaded.
     */
    public function update(DaftarRequest $request, Daftar $daftar): JsonResponse
    {
        $data = $request->safe()->except('berkas');

        if ($request->hasFile('berkas')) {
            if ($daftar->berkas) {
                Storage::disk('public')->delete($daftar->berkas);
            }

            $file = $request->file('berkas');
            $data['berkas'] = $file->store('berkas_pendaftaran', 'public');
            $data['nama_berkas'] = $file->getClientOriginalName();
        }

        $daftar->update($data);

        return response()->json(['success' => 'Data berhasil diperbarui.']);
    }

    /**
     * Remove the specified registration and its uploaded file.
     */
    public function destroy(Daftar $daftar): JsonResponse
    {
```

3.9 Model Daftar dan Validasi Form Pendaftaran

Model Daftar mendaftarkan semua kolom ke $fillable. Ada accessor berkas_url yang ikut di JSON lewat $appends, sehingga React cukup memakai URL siap pakai tanpa menyusun path sendiri. Factory Daftar didaftarkan lewat newFactory() supaya pengujian bisa memakai Daftar::factory().

Untitled-1

```php
<?php

namespace Modules\Pendaftaran\Models;

use Illuminate\Database\Eloquent\Casts\Attribute;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Modules\Pendaftaran\Database\Factories\DaftarFactory;

class Daftar extends Model
{
    /** @use HasFactory<DaftarFactory> */
    use HasFactory;

    /**
     * The attributes that are mass assignable.
     *
     * @var list<string>
     */
    protected $fillable = [
        'nama',
        'asal_sekolah',
        'prodi_tujuan',
        'email',
        'no_hp',
        'berkas',
        'nama_berkas',
    ];

    /**
     * The accessors to append to the model's array and JSON form.
     *
     * @var list<string>
     */
    protected $appends = [
        'berkas_url',
    ];

    /**
     * Public URL of the uploaded file, or null when there is no file.
     */
    protected function berkasUrl(): Attribute
    {
        return Attribute::get(fn (): ?string => $this->berkas
            ? asset('storage/'.$this->berkas)
            : null);
    }

    /**
     * Create a new factory instance for the model.
     */
    protected static function newFactory(): DaftarFactory
```

Validasi form pendaftaran ada di DaftarRequest. Semua isian teks wajib, email unik di tabel daftars, no_hp harus angka dengan panjang 8 sampai 20, dan berkas wajib saat menambah data tetapi boleh kosong saat mengedit. Jenis berkas dibatasi jpg, jpeg, png, atau pdf dengan ukuran maksimal 2 MB.

Untitled-1

```php
<?php

namespace Modules\Pendaftaran\Http\Requests;

use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Validation\Rule;

class DaftarRequest extends FormRequest
{
    /**
     * Get the validation rules that apply to the request.
     *
     * The file is required when creating, and optional when updating (keep the old file).
     *
     * @return array<string, array<int, mixed>>
     */
    public function rules(): array
    {
        $daftar = $this->route('daftar');

        return [
            'nama' => ['required', 'string', 'max:255'],
            'asal_sekolah' => ['required', 'string', 'max:255'],
            'prodi_tujuan' => ['required', 'string', 'max:255'],
            'email' => ['required', 'email', 'max:255', Rule::unique('daftars', 'email')->ignore($daftar?->id)],
            'no_hp' => ['required', 'string', 'regex:/^[0-9+]{8,20}$/'],
            'berkas' => [$daftar ? 'nullable' : 'required', 'file', 'mimes:jpg,jpeg,png,pdf', 'max:2048'],
        ];
    }

    /**
     * Get custom messages for validator errors.
     *
     * @return array<string, string>
     */
    public function messages(): array
    {
        return [
            'no_hp.regex' => 'No HP hanya boleh berisi angka (8-20 digit).',
            'berkas.required' => 'Berkas wajib diunggah.',
            'berkas.mimes' => 'Berkas harus berformat jpg, jpeg, png, atau pdf.',
            'berkas.max' => 'Ukuran berkas maksimal 2 MB.',
        ];
    }

    /**
     * Determine if the user is authorized to make this request.
     */
    public function authorize(): bool
    {
        return true;
    }
}
```

Pesan validasi saya tulis sendiri dalam bahasa Indonesia, misalnya No HP hanya boleh berisi angka (8-20 digit) dan Berkas wajib diunggah. Saya juga menambahkan lang/id/validation.php berisi terjemahan aturan bawaan Laravel beserta daftar nama atribut, karena sebelumnya pesan yang muncul masih berbahasa Inggris walaupun APP_LOCALE sudah diisi id.

3.10 Klien AJAX dan Halaman Pendaftaran

Untuk AJAX saya membuat helper http() di resources/js/lib/http.ts. Helper ini mengirim fetch dengan header Accept JSON, X-Requested-With, dan X-XSRF-TOKEN yang dibaca dari cookie XSRF-TOKEN. Respons yang tidak OK dilempar sebagai HttpError yang membawa status dan isi body, jadi halaman bisa membedakan validasi 422 dari error lain.



Halaman index Pendaftaran menyimpan data di state records dan memuatnya sekali lewat useEffect(). Pencarian dilakukan di browser dengan useMemo, jadi tidak ada request baru setiap kali mengetik. Form tambah dan edit memakai Dialog shadcn/ui dengan field yang dibangun dari array fields agar konsisten.

Untitled-1

```tsx
import { Head } from '@inertiajs/react';
import { FileImage, FileText, LoaderCircle, Pencil, Plus, Search, Trash2 } from 'lucide-react';
import { useEffect, useMemo, useState } from 'react';
import type { ChangeEvent, FormEvent } from 'react';
import { AlertMessage } from '@/components/alert-message';
import { ConfirmDialog } from '@/components/confirm-dialog';
import InputError from '@/components/input-error';
import { Badge } from '@/components/ui/badge';
import { Button } from '@/components/ui/button';
import {
    Dialog,
    DialogContent,
    DialogDescription,
    DialogFooter,
    DialogHeader,
    DialogTitle,
} from '@/components/ui/dialog';
import { Input } from '@/components/ui/input';
import { Label } from '@/components/ui/label';
import AppLayout from '@/layouts/app-layout';
import { http, HttpError } from '@/lib/http';
import type { ValidationErrors } from '@/lib/http';
import type { BreadcrumbItem } from '@/types';

type Daftar = {
    id: number;
    nama: string;
    asal_sekolah: string;
    prodi_tujuan: string;
    email: string;
    no_hp: string;
    berkas: string | null;
    nama_berkas: string | null;
    berkas_url: string | null;
};

type FormFields = Pick<Daftar, 'nama' | 'asal_sekolah' | 'prodi_tujuan' | 'email' | 'no_hp'>;
type Message = { type: 'success' | 'error'; text: string } | null;

const emptyForm: FormFields = { nama: '', asal_sekolah: '', prodi_tujuan: '', email: '', no_hp: '' };

const fields: { name: keyof FormFields; label: string; type?: string }[] = [
    { name: 'nama', label: 'Nama' },
    { name: 'asal_sekolah', label: 'Asal Sekolah' },
    { name: 'prodi_tujuan', label: 'Prodi Tujuan' },
    { name: 'email', label: 'Email', type: 'email' },
    { name: 'no_hp', label: 'No HP' },
];

const urls = {
    data: '/pendaftaran/data',
    store: '/pendaftaran',
    edit: (id: number) => `/pendaftaran/${id}/edit`,
    update: (id: number) => `/pendaftaran/${id}`,
    destroy: (id: number) => `/pendaftaran/${id}`,
};

const breadcrumbs: BreadcrumbItem[] = [{ title: 'Pendaftaran', href: '/pendaftaran' }];

// Tautan berkas yang dibuka di tab baru, ikon dipilih berdasarkan ekstensi
function FileLink({ url, name }: { url: string | null; name: string | null }) {
    if (!url) {
        return <span className="text-muted-foreground">-</span>;
    }

    const Icon = name?.toLowerCase().endsWith('.pdf') ? FileText : FileImage;

    return (
        <a
            href={url}
            target="_blank"
            rel="noopener noreferrer"
            className="inline-flex items-center gap-1 text-primary underline-offset-4 hover:underline"
        >
            <Icon className="size-4" /> {name ?? 'Lihat berkas'}
        </a>
    );
}
```

Saat form dikirim, semua isian dimasukkan ke FormData, berkas ditambahkan lewat formData.append('berkas', file), dan mode edit menambahkan _method=PUT. Kalau server menjawab 422, isi errors disimpan ke state dan ditampilkan per input lewat InputError. Kalau berhasil, dialog ditutup, pesan sukses muncul, dan data dimuat ulang tanpa berpindah halaman.

3.11 Komponen Pesan

Pesan berhasil dan error memakai satu komponen AlertMessage dengan dua varian warna. Varian untuk flash session ditangani FlashMessage, yang membaca shared prop flash dari Inertia dan menutup sendiri setelah empat detik. Halaman Pendaftaran memakai AlertMessage langsung karena pesannya berasal dari respons AJAX, bukan dari flash session.

Untitled-1

```tsx
import { CircleAlert, CircleCheck, X } from 'lucide-react';
import { useEffect, useState } from 'react';
import { useSharedProps } from '@/hooks/use-shared-props';

type AlertMessageProps = {
    type: 'success' | 'error';
    message?: string | null;
    onClose?: () => void;
};

const colors = {
    success: 'border-green-200 bg-green-50 text-green-800 dark:border-green-900 dark:bg-green-950 dark:text-green-200',
    error: 'border-red-200 bg-red-50 text-red-800 dark:border-red-900 dark:bg-red-950 dark:text-red-200',
};

export function AlertMessage({ type, message, onClose }: AlertMessageProps) {
    // Pesan berhasil otomatis tertutup setelah 4 detik
    useEffect(() => {
        if (!message || type !== 'success' || !onClose) {
            return;
        }

        const timer = setTimeout(onClose, 4000);

        return () => clearTimeout(timer);
    }, [message, type, onClose]);

    if (!message) {
        return null;
    }

    const Icon = type === 'success' ? CircleCheck : CircleAlert;

    return (
        <div role="alert" className={`flex items-start gap-3 rounded-lg border px-4 py-3 text-sm ${colors[type]}`}>
            <Icon className="mt-0.5 size-4 shrink-0" />
            <p className="flex-1">{message}</p>
            {onClose && (
                <button
                    type="button"
                    onClick={onClose}
                    className="opacity-70 hover:opacity-100"
                    aria-label="Tutup"
                >
                    <X className="size-4" />
                </button>
            )}
        </div>
    );
}

export function FlashMessage() {
    const { flash } = useSharedProps();
    const [success, setSuccess] = useState(flash.success);
    const [error, setError] = useState(flash.error);

    // Tampilkan ulang pesan setiap kali server mengirim flash baru
    useEffect(() => {
        setSuccess(flash.success);
        setError(flash.error);
    }, [flash]);

```

3.12 Sidebar dan Resolusi Halaman Modul

Menu sidebar menampilkan Dashboard dan Pendaftaran untuk semua pengguna yang login, sedangkan Kelola User hanya muncul untuk admin. Data role dibaca dari shared prop auth.user lewat hook useSharedProps().

Untitled-1

```tsx
import { Link } from '@inertiajs/react';
import { ClipboardList, LayoutGrid, Users } from 'lucide-react';
import AppLogo from '@/components/app-logo';
import { NavMain } from '@/components/nav-main';
import { NavUser } from '@/components/nav-user';
import {
    Sidebar,
    SidebarContent,
    SidebarFooter,
    SidebarHeader,
    SidebarMenu,
    SidebarMenuButton,
    SidebarMenuItem,
} from '@/components/ui/sidebar';
import { useSharedProps } from '@/hooks/use-shared-props';
import type { NavItem } from '@/types';

export function AppSidebar() {
    const { isAdmin } = useSharedProps();

    const mainNavItems: NavItem[] = [
        { title: 'Dashboard', url: '/dashboard', icon: LayoutGrid },
        { title: 'Pendaftaran', url: '/pendaftaran', icon: ClipboardList },
        ...(isAdmin ? [{ title: 'Kelola User', url: '/admin/users', icon: Users }] : []),
    ];

    return (
        <Sidebar collapsible="icon" variant="inset">
            <SidebarHeader>
                <SidebarMenu>
                    <SidebarMenuItem>
                        <SidebarMenuButton size="lg" asChild>
                            <Link href="/dashboard" prefetch>
                                <AppLogo />
                            </Link>
                        </SidebarMenuButton>
                    </SidebarMenuItem>
                </SidebarMenu>
            </SidebarHeader>
            <SidebarContent>
                <NavMain items={mainNavItems} />
            </SidebarContent>
            <SidebarFooter>
                <NavUser />
            </SidebarFooter>
        </Sidebar>
    );
}
```

Halaman modul dirender dengan nama UserManagement::index atau Pendaftaran::index. Nama tersebut belum dikenali oleh import.meta.glob bawaan starter kit, jadi saya menambahkan glob kedua untuk folder Modules/*/resources/js/pages dan memecah nama halaman pada tanda :: sebelum memanggil resolvePageComponent().

Untitled-1

```
import '../css/app.css';

import { createInertiaApp } from '@inertiajs/react';
import { resolvePageComponent } from 'laravel-vite-plugin/inertia-helpers';
import { createRoot } from 'react-dom/client';
import { route as routeFn } from 'ziggy-js';
import { initializeTheme } from './hooks/use-appearance';

declare global {
    const route: typeof routeFn;
}

const appName = import.meta.env.VITE_APP_NAME || 'Laravel';

createInertiaApp({
    title: (title) => `${title} - ${appName}`,
    resolve: (name) => {
        const pages = import.meta.glob(['./pages/**/*.tsx',
'../../Modules/*/resources/js/pages/**/*.tsx']);

        // Halaman modul ditulis dengan format NamaModul::nama-halaman
        if (name.includes('::')) {
            const [module, page] = name.split('::');

            return
resolvePageComponent(`../../Modules/${module}/resources/js/pages/${page}.tsx`, pages);
        }

        return resolvePageComponent(`./pages/${name}.tsx`, pages);
    },
    setup({ el, App, props }) {
        const root = createRoot(el);

        root.render(<App {...props} />);
    },
    progress: {
        color: '#4B5563',
    },
});

// This will set light / dark mode on load...
initializeTheme();
```

3.13 Data Bersama dan Dashboard

HandleInertiaRequests membagikan auth.user dan flash success/error ke setiap halaman. Karena objek User sudah di-cast, role ikut terkirim sebagai string admin atau user, sehingga React tidak perlu menebak.

Untitled-1

```php
<?php

namespace App\Http\Middleware;

use Illuminate\Foundation\Inspiring;
use Illuminate\Http\Request;
use Inertia\Middleware;

class HandleInertiaRequests extends Middleware
{
    /**
     * The root template that's loaded on the first page visit.
     *
     * @see https://inertiajs.com/server-side-setup#root-template
     *
     * @var string
     */
    protected $rootView = 'app';

    /**
     * Determines the current asset version.
     *
     * @see https://inertiajs.com/asset-versioning
     */
    public function version(Request $request): ?string
    {
        return parent::version($request);
    }

    /**
     * Define the props that are shared by default.
     *
     * @see https://inertiajs.com/shared-data
     *
     * @return array<string, mixed>
     */
    public function share(Request $request): array
    {
        [$message, $author] = str(Inspiring::quotes()->random())->explode('-');

        return [
            ...parent::share($request),
            'name' => config('app.name'),
            'quote' => ['message' => trim($message), 'author' => trim($author)],
            'auth' => [
                'user' => $request->user(),
            ],
            'flash' => [
                'success' => fn () => $request->session()->get('success'),
                'error' => fn () => $request->session()->get('error'),
            ],
        ];
    }
}
```

Hook useSharedProps() merangkum auth.user, isAdmin, dan flash supaya komponen tidak menulis ulang tipe yang sama. DashboardController menghitung jumlah pendaftar untuk semua pengguna, sedangkan total user hanya dikirim ke admin. Kartu jumlah user tidak tampil bagi user biasa, dan route-nya tetap aman karena perlindungan sebenarnya berada di middleware dan Form Request.

Untitled-1

```tsx
import { usePage } from '@inertiajs/react';

export type UserRole = 'admin' | 'user';

export type AuthUser = {
    id: number;
    name: string;
    email: string;
    role: UserRole;
};

type SharedProps = {
    auth: { user: AuthUser };
    flash: { success?: string | null; error?: string | null };
};

export function useSharedProps() {
    const { auth, flash } = usePage<SharedProps>().props;

    return {
        user: auth.user,
        isAdmin: auth.user?.role === 'admin',
        flash,
    };
}
```

Untitled-1

```php
<?php

namespace App\Http\Controllers;

use App\Models\User;
use Illuminate\Http\Request;
use Inertia\Inertia;
use Inertia\Response;
use Modules\Pendaftaran\Models\Daftar;

class DashboardController extends Controller
{
    /**
     * Show the dashboard with record counts per module.
     */
    public function __invoke(Request $request): Response
    {
        return Inertia::render('dashboard', [
            'totalPendaftar' => Daftar::count(),
            'totalUsers' => $request->user()?->isAdmin() ? User::count() : null,
        ]);
    }
}
```

3.14 Seeder

DatabaseSeeder mengisi dua akun awal: Administrator dengan email admin@example.com berrole admin, dan User Biasa dengan email user@example.com berrole user. Keduanya memakai password yang sama untuk keperluan praktikum. Data pendaftar tidak di-seed karena diisi lewat halaman AJAX.

Untitled-1

```php
<?php

namespace Database\Seeders;

use App\Models\User;
use Illuminate\Database\Console\Seeds\WithoutModelEvents;
use Illuminate\Database\Seeder;

class DatabaseSeeder extends Seeder
{
    use WithoutModelEvents;

    /**
     * Seed the application's database.
     */
    public function run(): void
    {
        User::factory()->admin()->create([
            'name' => 'Administrator',
            'email' => 'admin@example.com',
            'password' => 'password',
        ]);

        User::factory()->create([
            'name' => 'User Biasa',
            'email' => 'user@example.com',
            'password' => 'password',
        ]);
    }
}
```

3.15 Menjalankan Aplikasi dan Pengujian Otomatis

Aplikasi dijalankan dengan npm run dev di terminal pertama dan php artisan serve di terminal kedua, lalu dibuka di http://127.0.0.1:8000. Untuk lingkungan pengujian, phpunit.xml memakai SQLite :memory:, jadi tes tidak menyentuh database hmvc_react. Berkas tes yang saya tambahkan adalah tests/Feature/RoleTest.php, tests/Feature/UserManagementTest.php,

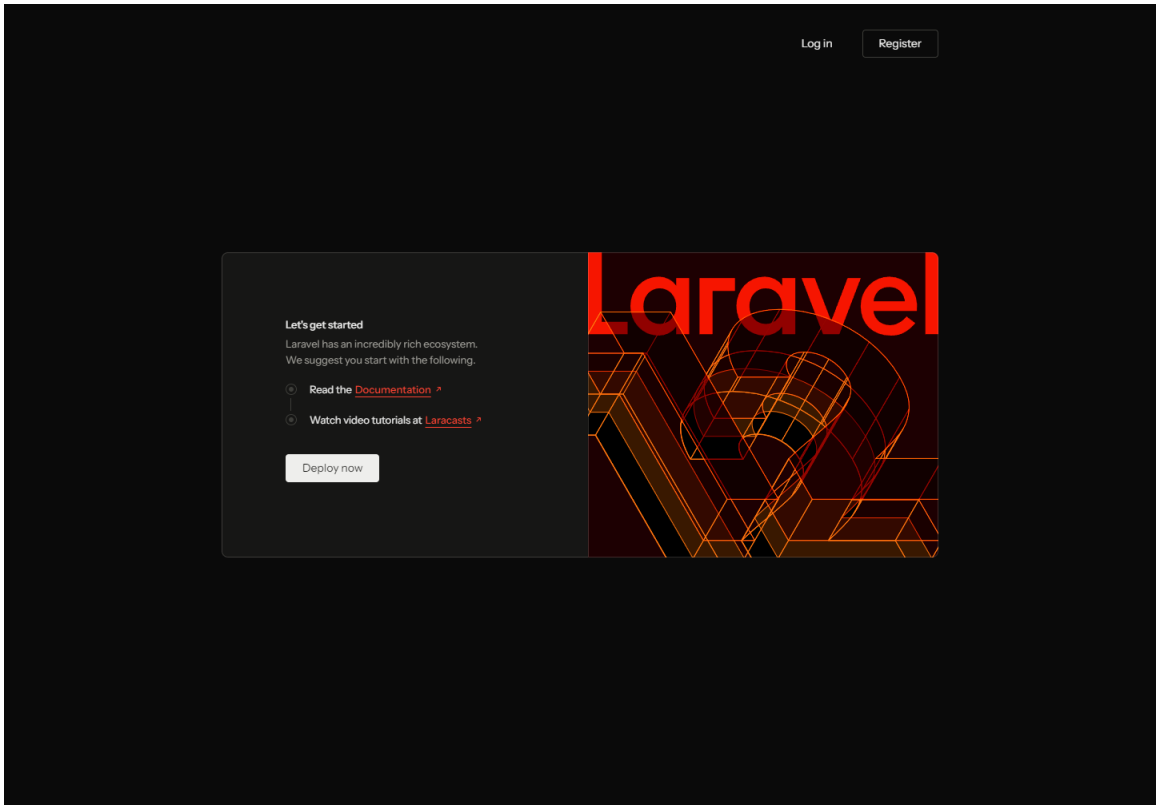
tests/Feature/PendaftaranTest.php, tests/Feature/DashboardTest.php, dan tests/Feature/DatabaseSeederTest.php. Seluruh tes dijalankan dengan php artisan test.

Tes modul Pendaftaran memakai Storage::fake('public') dan UploadedFile::fake(), sehingga berkas tidak benar-benar ditulis ke storage. Tes route modul memakai ->component('Modul::page', shouldExist: false) karena view finder Inertia menyelesaikan nama ber-namespace ke folder pages biasa. Setiap tes yang butuh pengguna login memakai $this->actingAs() langsung, karena helper yang memanggil test()->actingAs() di dalam closure ternyata tidak mengautentikasi request yang sedang diuji.

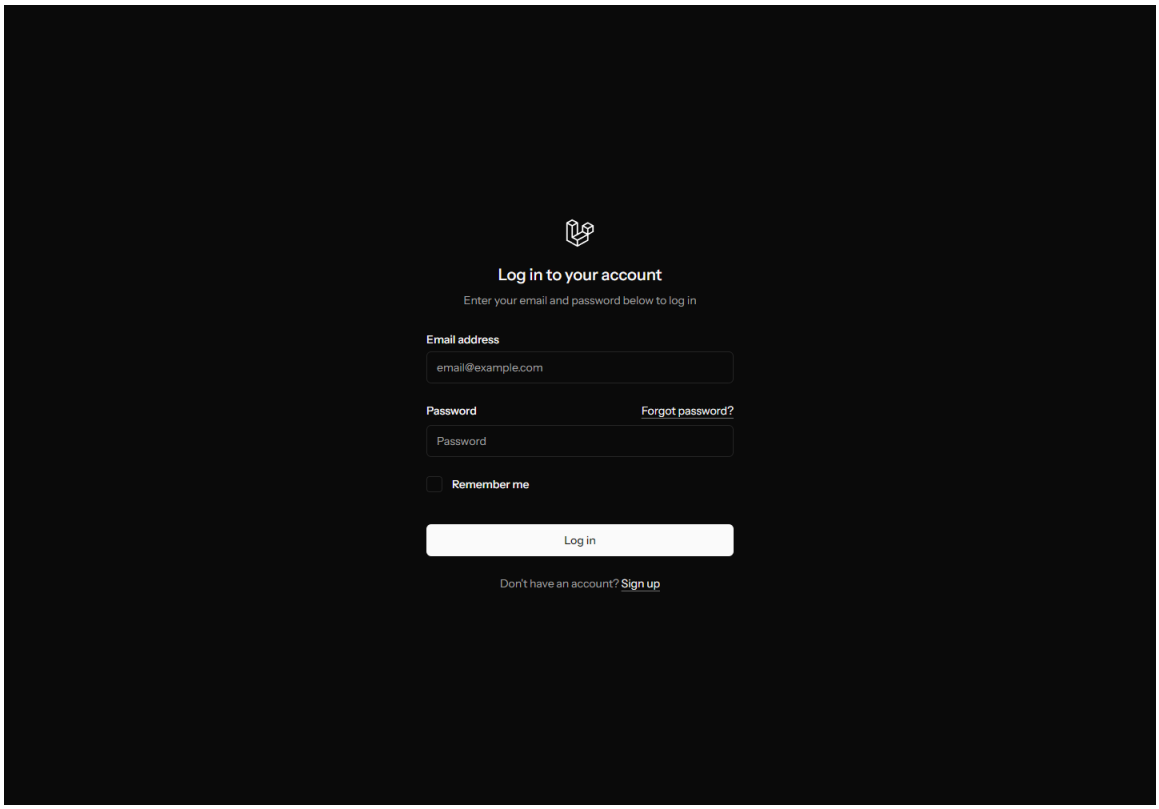
4. Hasil dan Dokumentasi Program

Semua bagian sudah berjalan. Tabel users memiliki kolom role, middleware role terpasang pada route kedua modul, dan halaman kelola user maupun pendaftaran sudah tampil sebagai komponen React. Pengujian otomatis menghasilkan 75 tes lulus dengan 338 assertion dalam waktu sekitar lima detik.

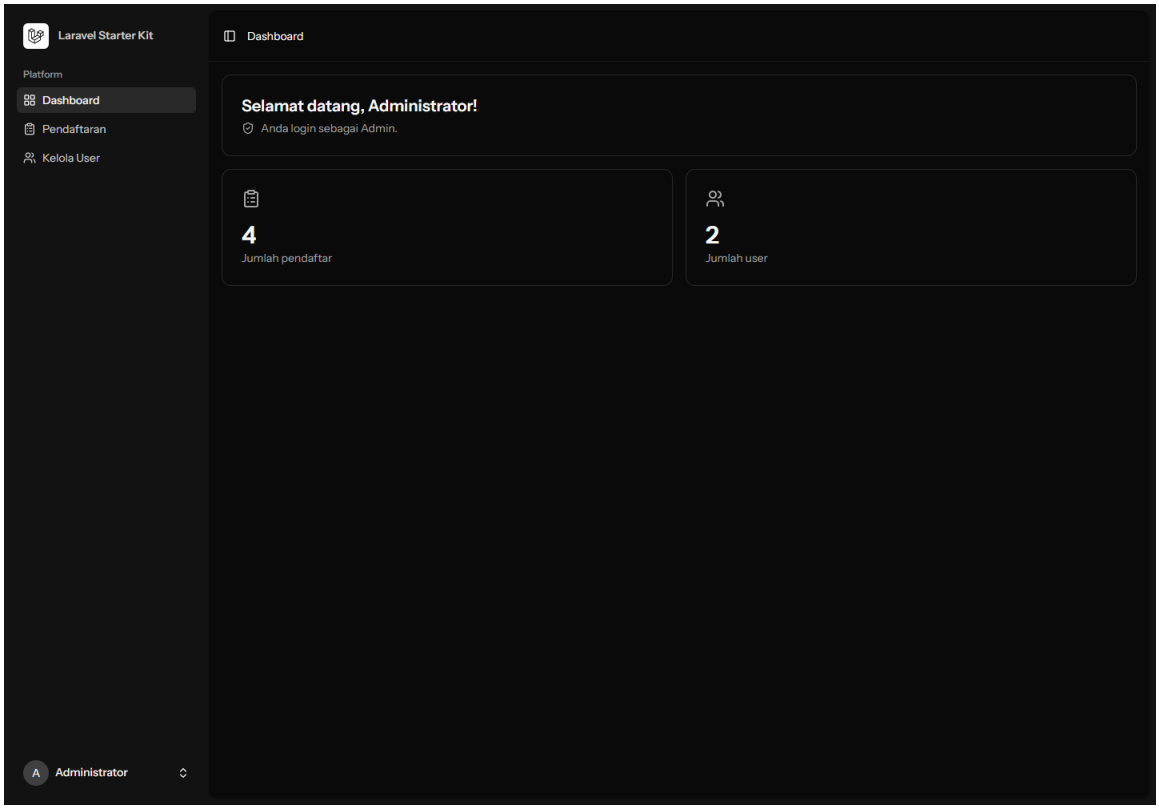
Gambar di bawah ini saya ambil dari aplikasi yang benar-benar berjalan di http://127.0.0.1:8123 dengan data seeder dan beberapa data pendaftar. Tampilan memakai tema gelap bawaan starter kit.



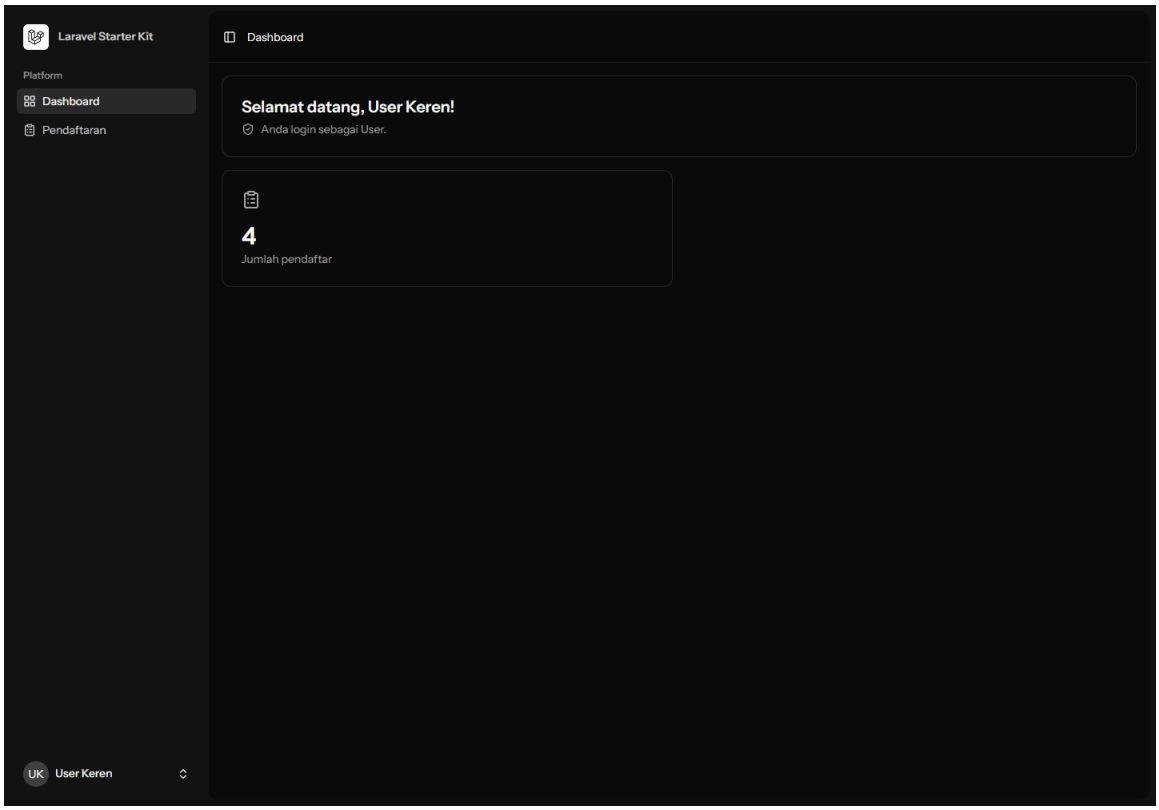
Gambar 1. Halaman beranda starter kit sebelum login.



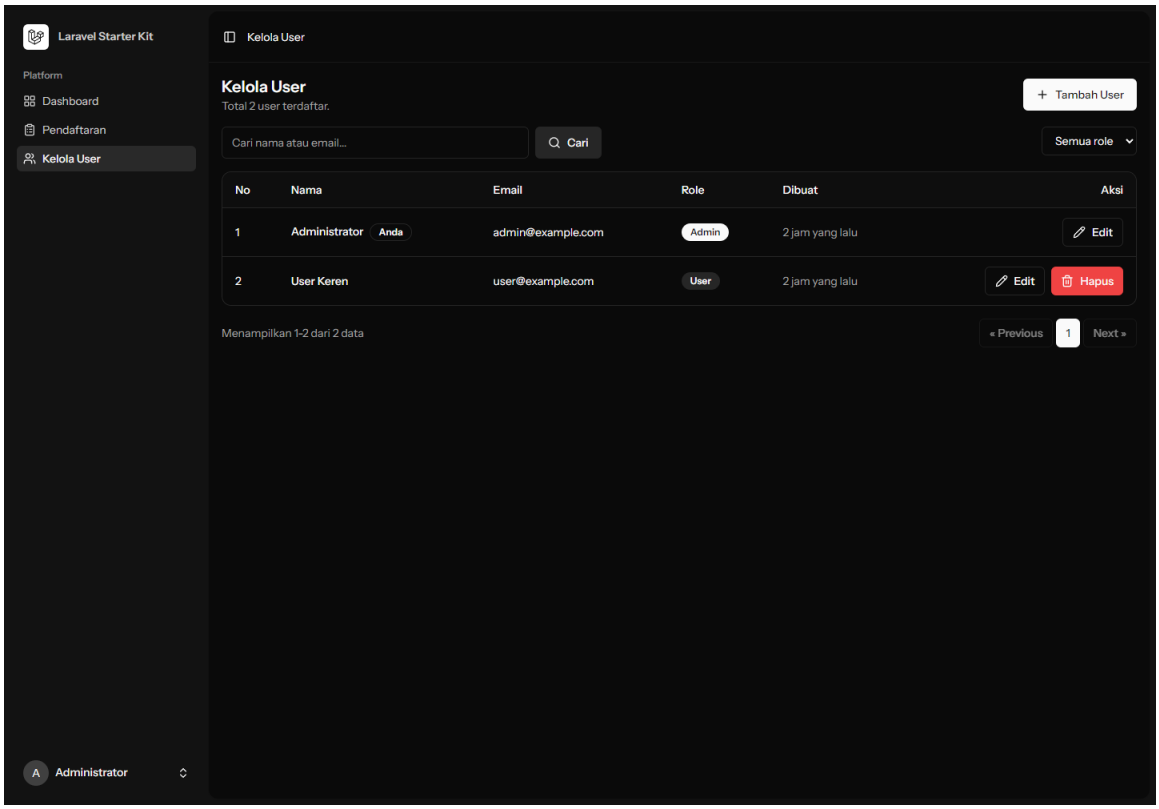
Gambar 2. Halaman login bawaan starter kit.



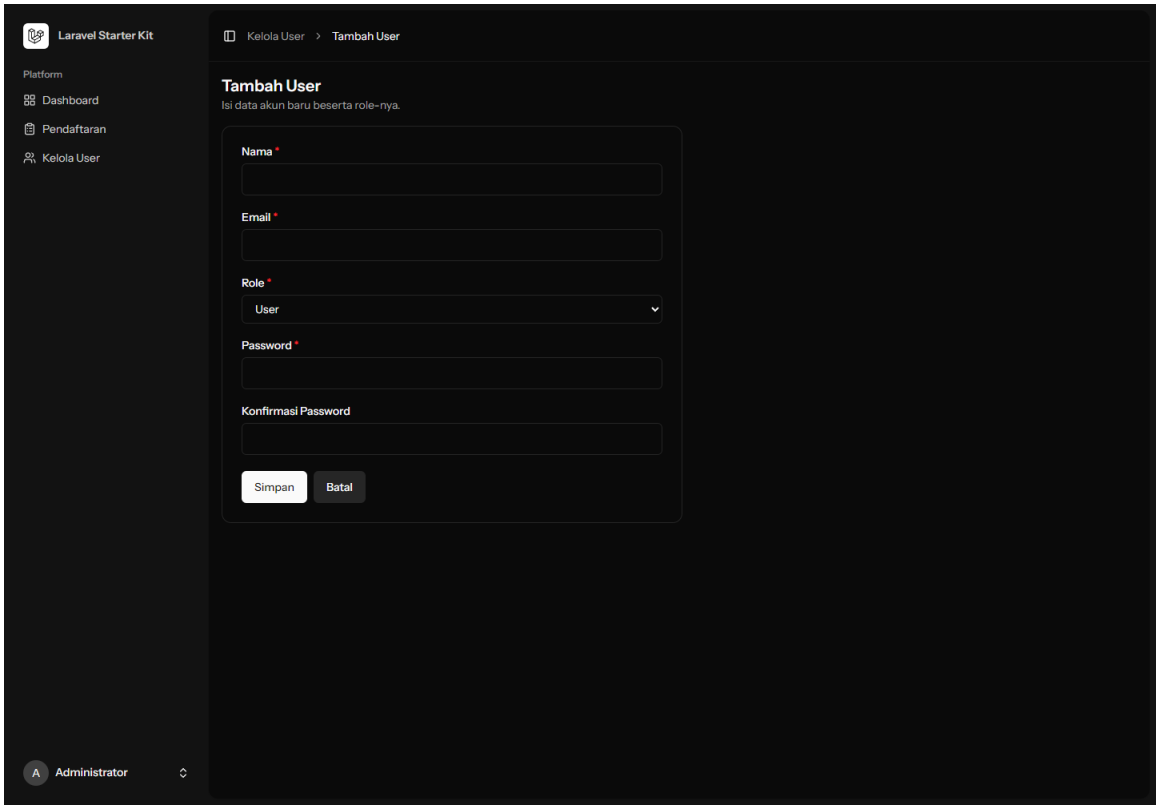
Gambar 3. Dashboard setelah login sebagai admin, dengan menu Kelola User dan kartu jumlah user.



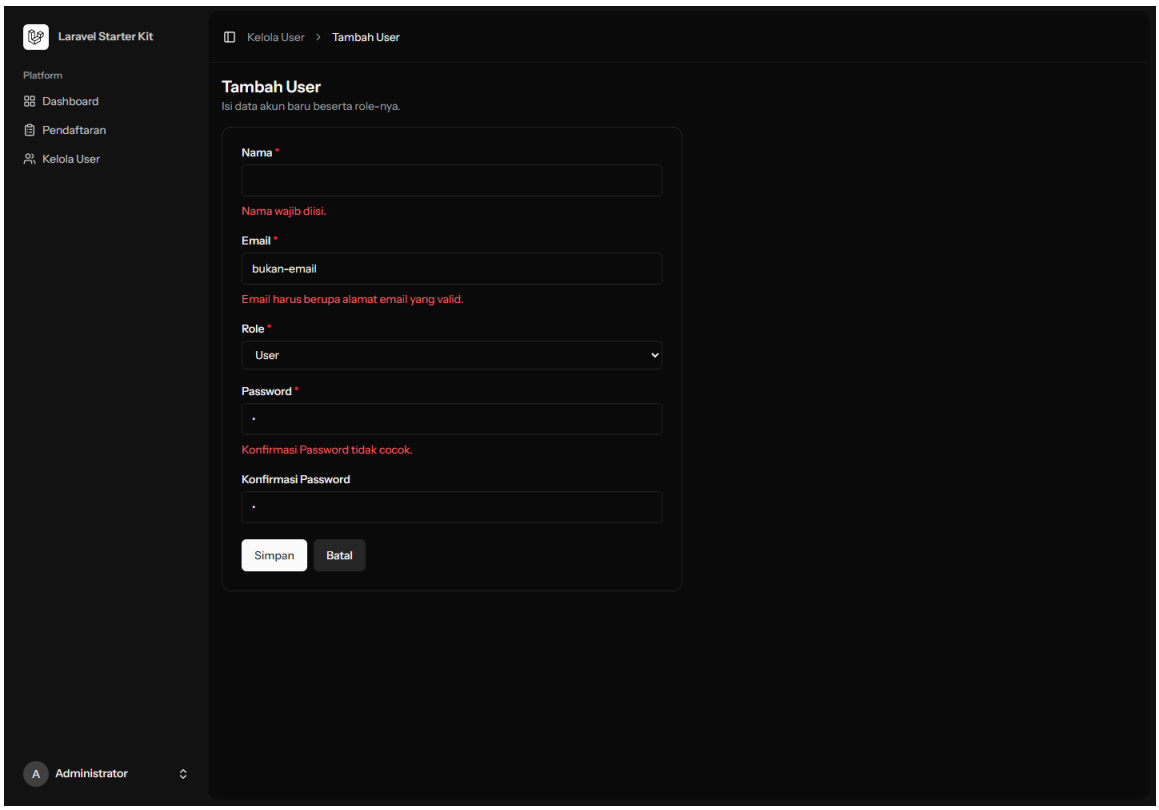
Gambar 4. Dashboard sebagai user biasa. Menu Kelola User dan kartu jumlah user tidak tampil.



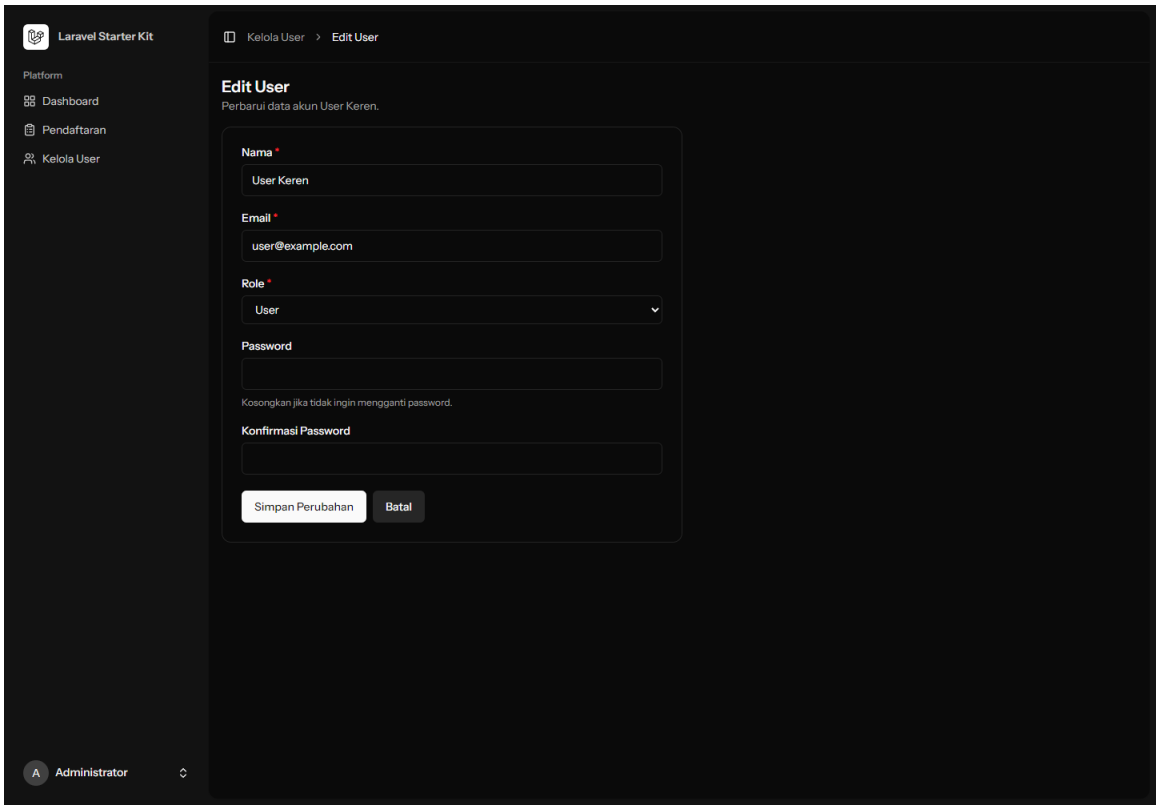
Gambar 5. Halaman Kelola User dengan pencarian, filter role, dan pagination.



Gambar 6. Form tambah user dengan pilihan role dari enum UserRole.



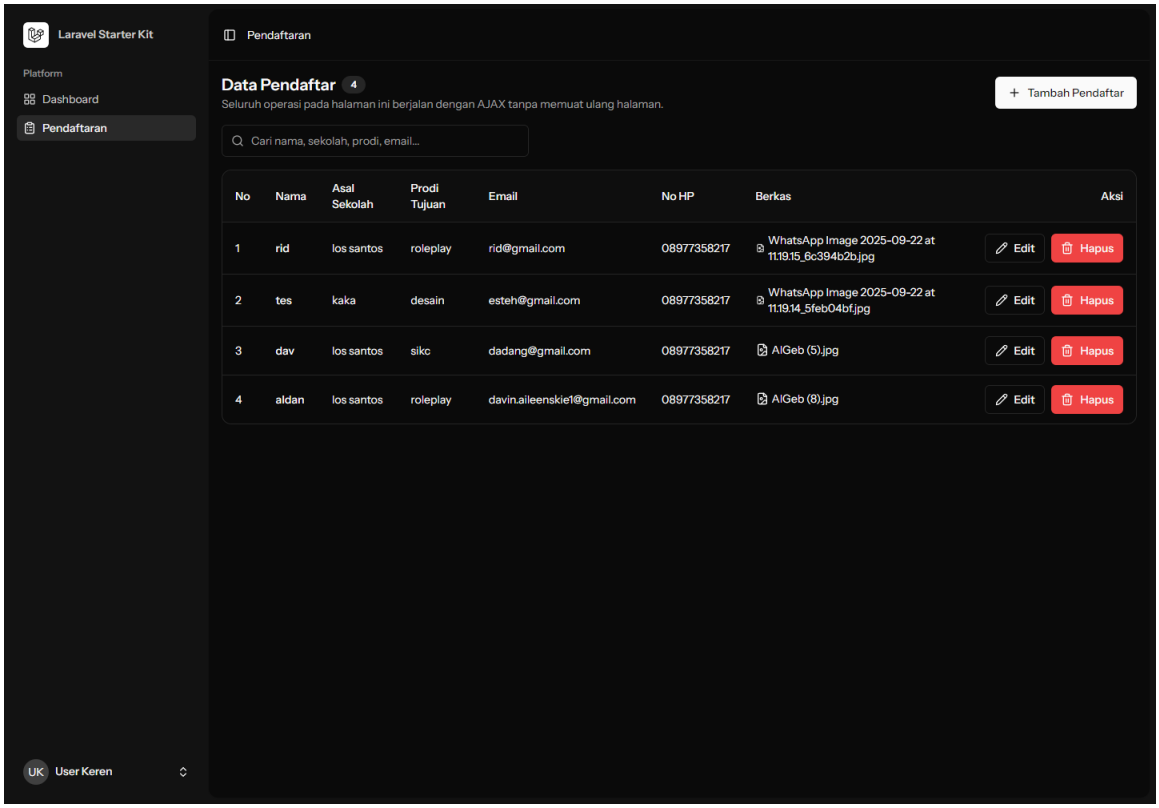
Gambar 7. Pesan validasi per input pada form tambah user, tampil dalam bahasa Indonesia.



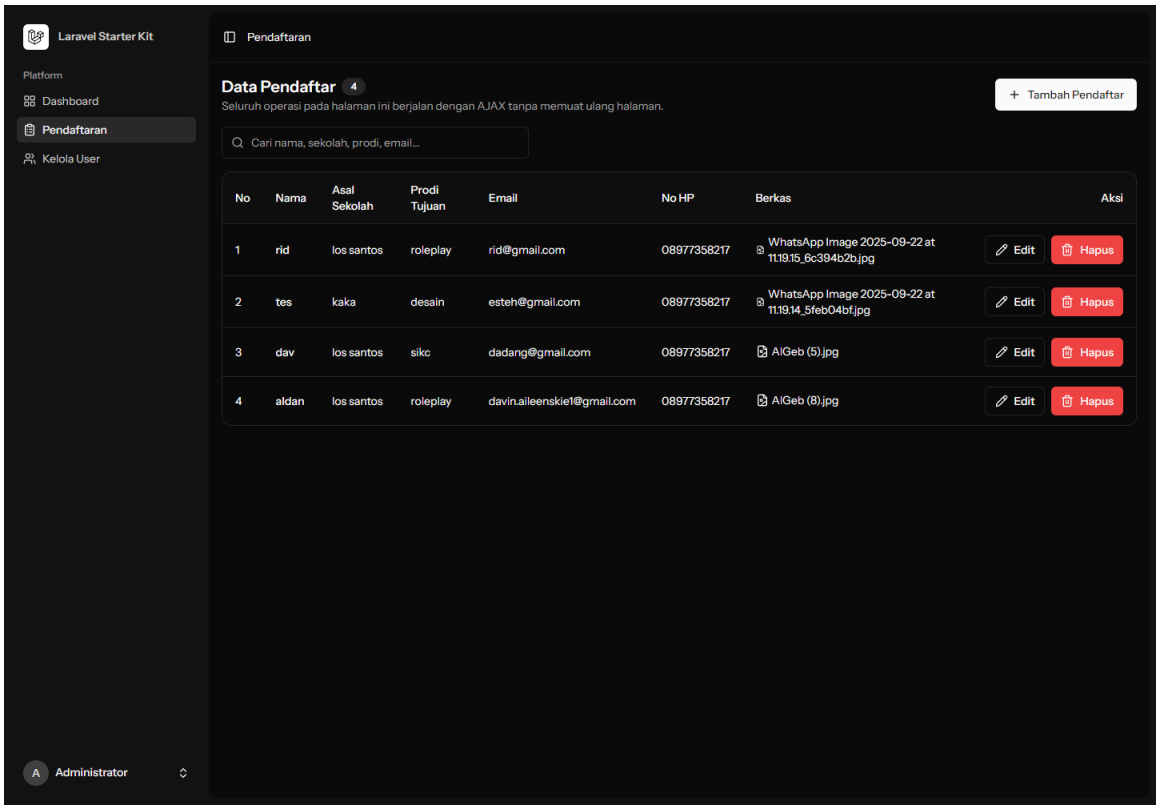
Gambar 8. Form edit user. Kolom password boleh kosong bila tidak ingin diganti.



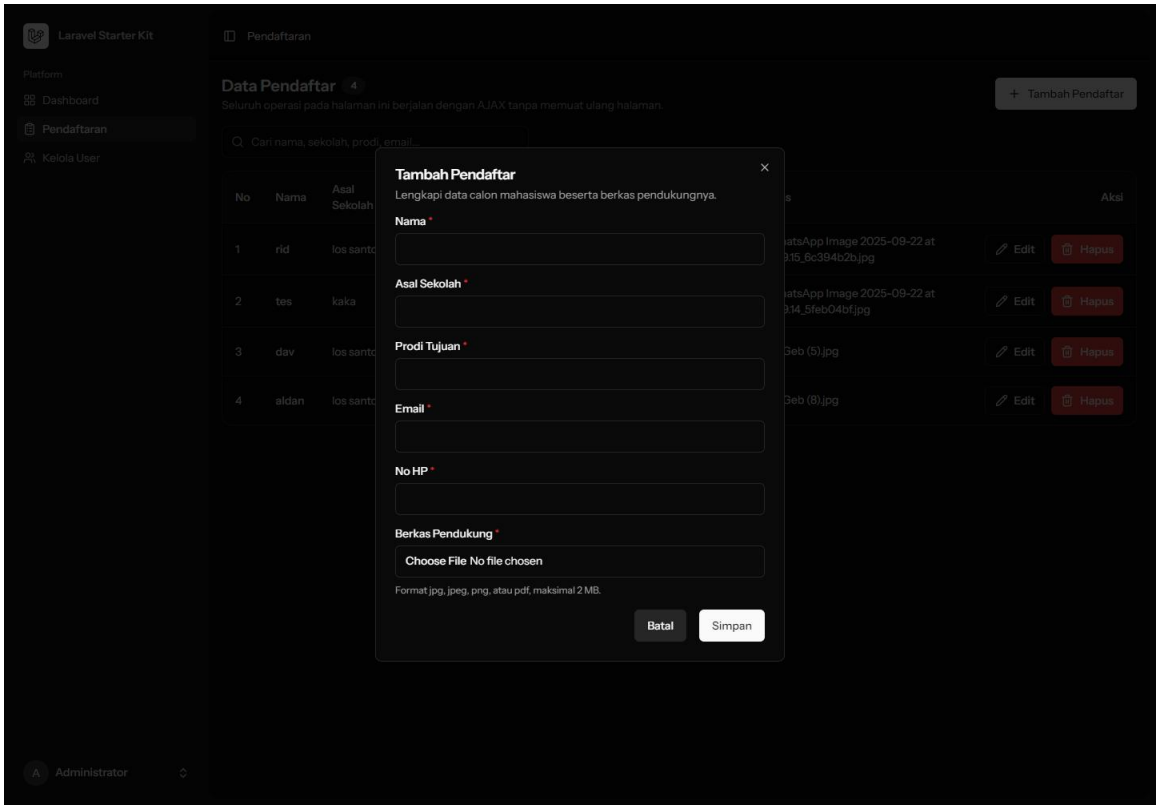
Gambar 9. Halaman 403 saat user biasa membuka /admin/users.



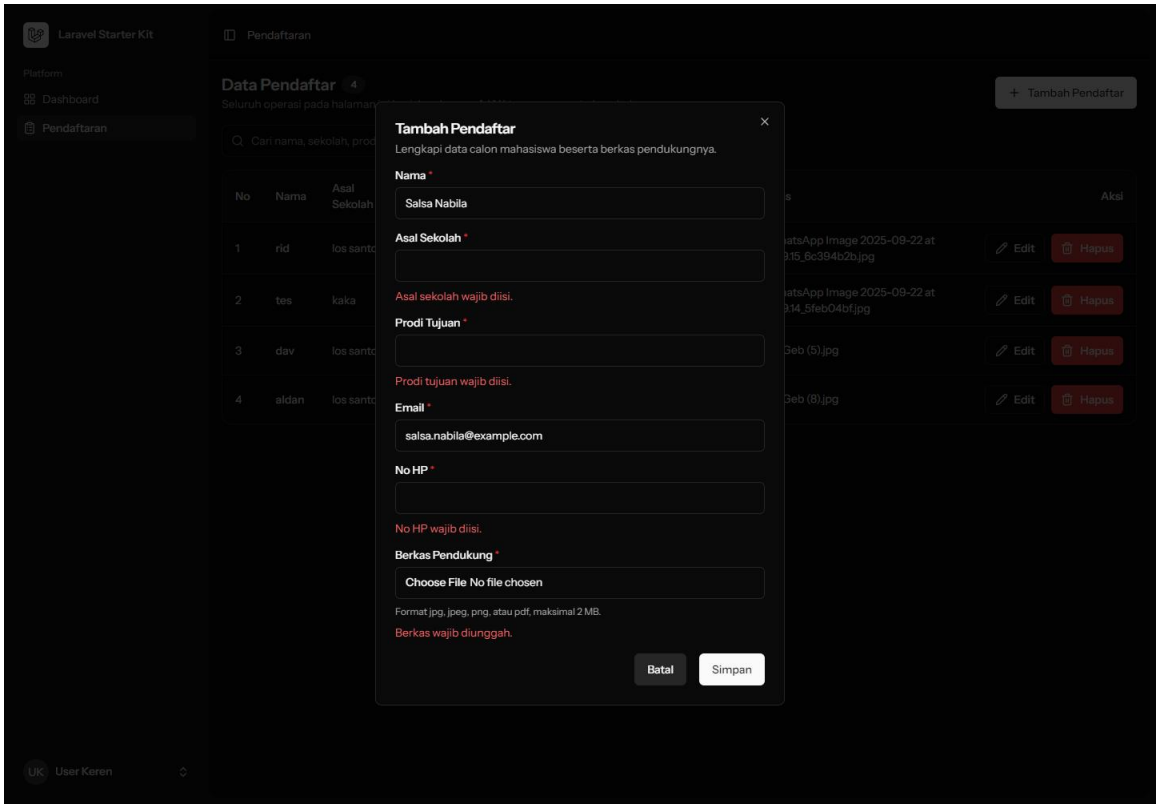
Gambar 10. Halaman Pendaftaran ketika dibuka oleh user biasa. Menu Kelola User tidak ada di sidebar.



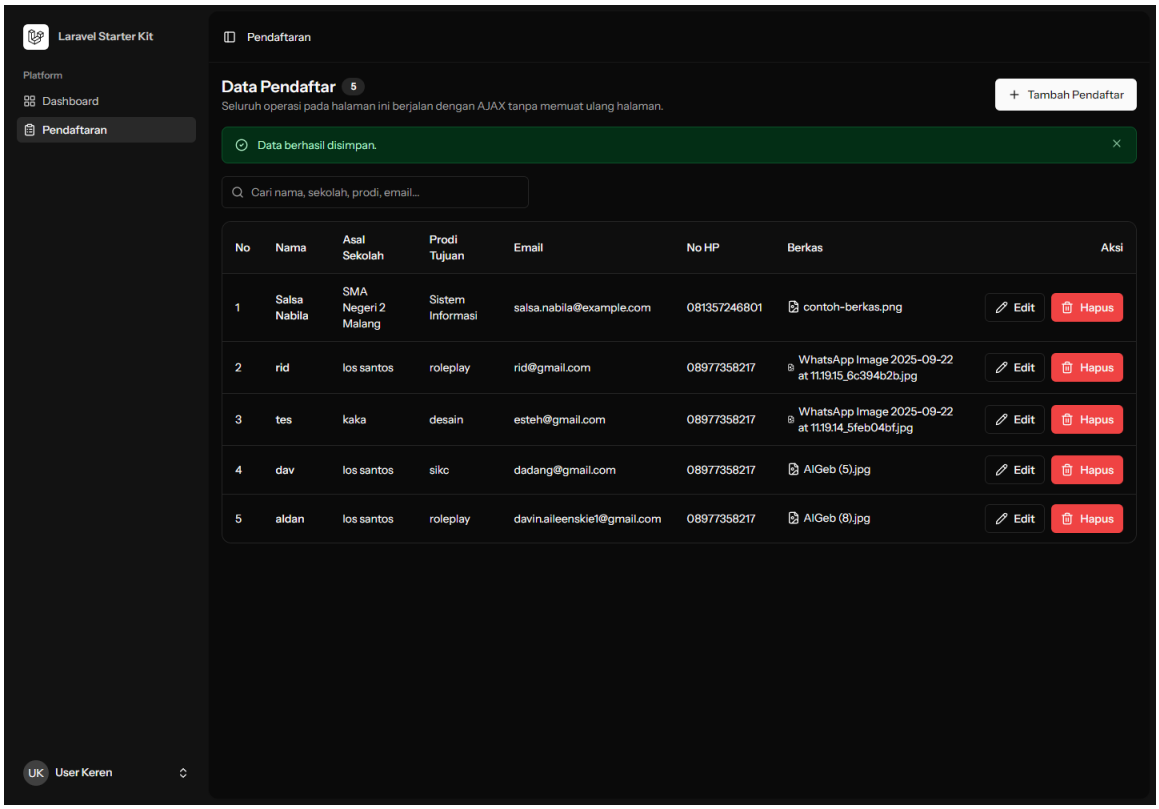
Gambar 11. Halaman Pendaftaran yang memuat data lewat AJAX.



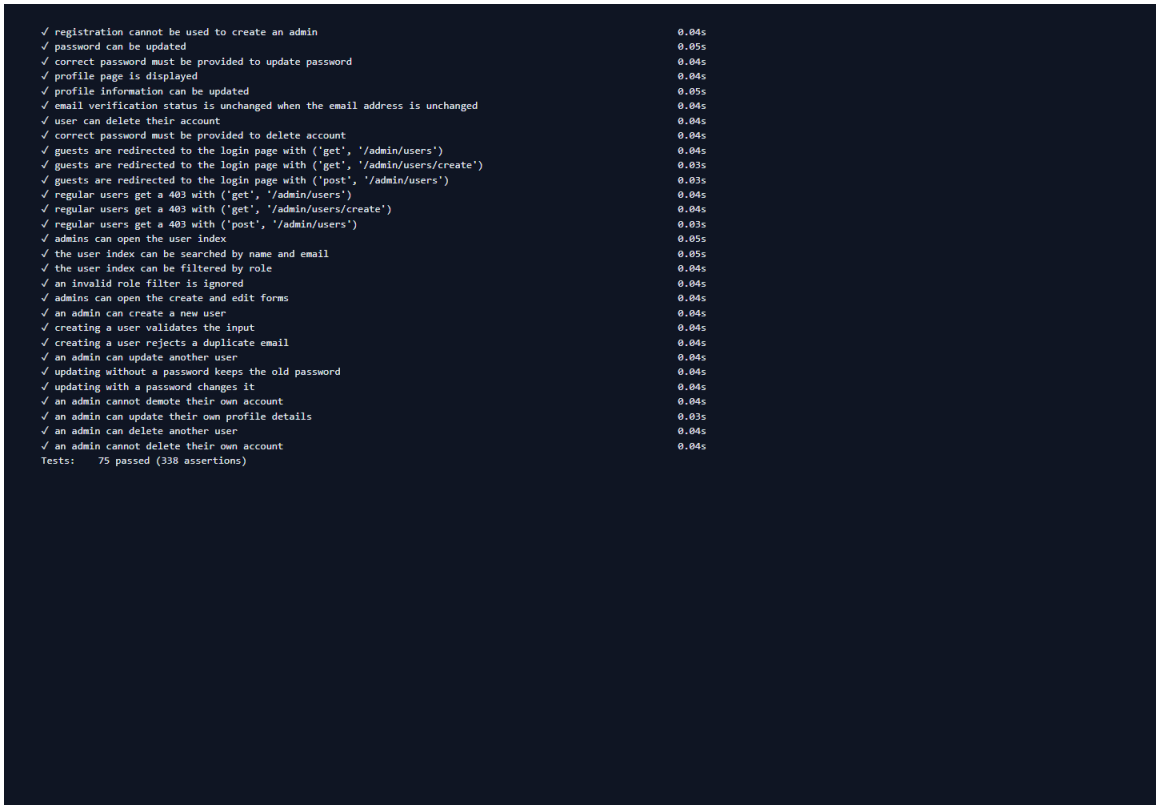
Gambar 12. Dialog tambah pendaftar beserta kolom unggah berkas.



Gambar 13. Respons 422 dari AJAX ditampilkan sebagai pesan error di bawah tiap field.



Gambar 14. Data baru muncul di tabel tanpa memuat ulang halaman, disertai pesan berhasil.



Gambar 15. Hasil php artisan test, seluruh berkas tes lulus.

5. Analisis dan Pembahasan

1. Pemetaan HMVC pada project ini.

HMVC di sini bukan berarti setiap modul berjalan sebagai aplikasi terpisah. Semua modul masih memakai satu database, satu konfigurasi, dan satu proses Laravel. Yang dipisahkan adalah kode: controller, request, model, route, dan halaman React milik UserManagement dan Pendaftaran berada di dalam folder Modules, sementara autentikasi, layout, dan komponen UI tetap di app dan resources. Dependency antarmodul nyaris tidak ada karena keduanya hanya berbagi model User dan komponen tampilan starter kit.

2. Autentikasi di luar modul, otorisasi di dalam route.

Autentikasi tetap memakai controller bawaan starter kit pada routes/auth.php. Saya menambahkan require __DIR__.'/auth.php'; di routes/web.php karena tanpanya route login tidak terdaftar dan halaman beranda mengarah ke 404. Otorisasi baru muncul setelah pengguna dikenali, yaitu lewat middleware role pada grup route modul. Pemisahan ini membuat aturan akses mudah dibaca: cukup melihat deklarasi grup route untuk tahu siapa yang boleh masuk.

3. Pembatasan akses berlapis.

Middleware role:admin adalah penjaga pertama. Authorize() pada StoreUserRequest dan UpdateUserRequest adalah lapisan kedua untuk operasi tulis, sedangkan lapisan ketiga ada di tampilan: menu Kelola User hanya dirender untuk admin dan tombol hapus disembunyikan pada baris akun sendiri. Lapisan tampilan bukan pengaman utama, tetapi mengurangi peluang pengguna menekan aksi yang pasti ditolak server.

4. Pengecekan role yang aman terhadap nilai tidak dikenal.

Filter role di halaman kelola user memakai UserRole::tryFrom(). Nilai yang tidak dikenal akan menjadi null dan filter dilewati, bukan menghasilkan error. Ada tes khusus yang mengirim role=superadmin dan memastikan halaman tetap terbuka dengan seluruh data. Ini penting karena parameter filter datang dari URL yang bisa diubah siapa saja.

5. Perbandingan CRUD Inertia dan CRUD AJAX.

CRUD user memakai Inertia seperti halaman lain di starter kit: form mengirim request, controller mengembalikan redirect, dan Inertia menukar komponen tanpa memuat ulang dokumen. Halaman Pendaftaran memakai cara yang lebih manual: fetch() ke endpoint JSON, state React diperbarui, dan tabel digambar ulang. Cara kedua lebih banyak menulis kode, tetapi jelas memperlihatkan alur request dan respons yang dimaksud modul.

6. Token CSRF pada permintaan AJAX.

fetch() tidak otomatis mengirim token CSRF seperti form Laravel. Helper http() membaca cookie XSRF-TOKEN yang dibuat Laravel dan mengirimkannya sebagai header X-XSRF-TOKEN. Saat menguji dengan skrip terpisah, permintaan tanpa header ini langsung dijawab 419, jadi jelas

bahwa token memang diperlukan. Hal yang sama berlaku untuk update: metode PUT dikirim lewat POST dengan _method=PUT karena FormData tidak mendukung PUT secara langsung.

7. Pengelolaan berkas pendaftaran.

Berkas disimpan di storage/app/public/berkas_pendaftaran dengan nama acak dari store(), sedangkan nama asli disimpan di kolom nama_berkas. Tautan berkas dibentuk oleh accessor berkas_url memakai asset('storage/...'), sehingga butuh php artisan storage:link agar bisa dibuka. Saat berkas diganti, berkas lama dihapus lalu berkas baru disimpan dengan nama acak. Saat data dihapus, berkasnya ikut dihapus supaya storage tidak menumpuk file yatim.

8. Kendala dan penyelesaian.

Kendala pertama muncul saat menjalankan pengujian: tes verifikasi email bawaan starter kit gagal karena redirect-nya mengarah ke /home, bukan /dashboard. Setelah saya telusuri, rute verify-email milik Fortify ternyata terdaftar lebih dulu dan menimpa rute starter kit, sehingga response Fortify dengan konfigurasi home=/home yang dipakai. Perbaikannya adalah memanggil Fortify::ignoreRoutes() di AppServiceProvider dan menambahkan config/fortify.php berisi home=/dashboard. Setelah itu seluruh tes lulus.

Kendala kedua ada di tampilan. Halaman modul mengembalikan error Unable to locate file in Vite manifest karena resources/views/app.blade.php memuat @vite untuk halaman per halaman, padahal hanya bundle app.tsx yang dibangun. Saya menggantinya dengan satu @vite(['resources/js/app.tsx']) dan mengandalkan Inertia untuk merender komponen. Kendala ketiga, pesan validasi masih berbahasa Inggris walaupun APP_LOCALE=id, karena lang/id hanya berisi auth.php. Solusinya menambahkan lang/id/validation.php beserta nama atribut.

Kendala keempat saya temukan saat mengambil tangkapan layar. Submit dengan fetch() biasa ternyata tidak menampilkan pesan validasi di halaman, karena respons error tidak diteruskan ke komponen. Setelah form diisi dan dikirim lewat tombol di dalam halaman, pesan per input muncul seperti pada Gambar 7 dan Gambar 13. Ini sekaligus memastikan alur yang dipakai pengguna benar-benar bekerja.

9. Perbedaan dengan modul.

Perbedaan paling besar ada pada basis aplikasinya. Modul memakai Laravel dengan Blade, Bootstrap, dan jQuery, sedangkan saya memakai Laravel React Starter Kit dengan Inertia, React, dan shadcn/ui. Akibatnya semua bagian tampilan ditulis sebagai komponen TypeScript, modal memakai Dialog shadcn/ui, dan pesan error memakai komponen InputError. Di sisi server, pola yang dipakai tetap sama seperti modul: migration terpisah, enum role, middleware dengan parameter, Form Request, serta controller yang memisahkan halaman dan endpoint JSON. Perbedaan lain, project ini tidak memakai fitur verifikasi email pada model User karena baris MustVerifyEmail di model memang dikomentari secara bawaan starter kit.

6. Kesimpulan

Struktur modular berhasil diterapkan pada Laravel React Starter Kit dengan paket nwidart/laravel-modules. Modul UserManagement dan Pendaftaran berdiri sebagai folder

terpisah dengan route, controller, request, model, dan halaman React sendiri, sementara autentikasi dan layout tetap berada di luar modul.

Otorisasi berbasis role berjalan di dua tingkat. Middleware role:admin dan role:admin,user menjaga route, sedangkan Form Request menjaga operasi tulis. Pengujian membuktikan pengunjung diarahkan ke halaman login, user biasa menerima 403 saat membuka /admin/users, dan admin tidak bisa menurunkan role atau menghapus akunnya sendiri.

CRUD AJAX di modul Pendaftaran berjalan tanpa memuat ulang halaman, termasuk unggah dan penggantian berkas. Respons 422 dari server tampil sebagai pesan error tepat di bawah field yang bermasalah, dan respons berhasil tampil sebagai alert di atas tabel. Pesan validasi sudah berbahasa Indonesia setelah lang/id/validation.php ditambahkan.

Pengujian otomatis dengan Pest mencakup enum dan middleware role, CRUD kedua modul, unggah berkas, seeder, dashboard, serta route bawaan starter kit. Hasil akhirnya 75 tes lulus dengan 338 assertion. Perintah php artisan test bisa dijalankan ulang kapan saja tanpa menyentuh database hmvc_react karena tes memakai SQLite di memori.

7. Link Repository Github

https://github.com/davinaileenskie1-hue/PABW-Semester-3